

NoTraffic Edge Sr. Software Engineer — Home Assignment

Take-home, Part 1 of 2. Suggested time: ~2.5 hours.

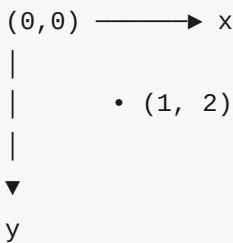
Background

Collecting statistics on red-light-runners is important for traffic engineers. To classify a vehicle as a red-light-runner, we need to detect the exact **time** at which the vehicle's center crossed the stop-line.

A **stop-line** is a line segment defined by two image points.

A **trajectory** is a list of `(t, x, y)` triplets — the timestamped image coordinates of the center of the vehicle's bounding box.

Coordinate system: image coordinates with `(0, 0)` at the top-left, `x` increasing to the right, `y` increasing downward. Coordinates are measured in **pixels**.



Goal

Implement a single class with one method that, given a stop-line and a trajectory, returns the **timestamp at which the vehicle crossed the stop-line segment** — or `None` if it didn't.

```
@dataclass(frozen=True)
class RLRequest:
    car_id: str
    stop_line: tuple[tuple[float, float], tuple[float, float]]
    trajectory: list[tuple[float, float, float]] # (t, x, y)

@dataclass(frozen=True)
class RLResult:
    car_id: str
    crossing_timestamp: float | None

class RLRequestProcessor:
    def process_request(self, request: RLRequest) -> RLResult: ...
```

Definition: when is a point "on the line"

A trajectory point is considered **on the stop-line** if its Euclidean distance to the stop-line segment is one pixel or less (≤ 1.0). This single tolerance — not exact floating-point equality — defines collinearity throughout this spec. A point farther than one pixel from the segment lies strictly on one side or the other.

What "crossing" means

A **crossing** occurs if and only if **both** of the following hold:

1. The trajectory visits **both** strict sides of the stop-line (it does not merely reach the line and stop or retreat); **and**
2. The transition point lies **on the segment** (within the 1-pixel tolerance) — not on the infinite extension beyond the endpoints.

A vehicle whose center only reaches the line and never proceeds to the far side has **not** crossed. Likewise, a path that swings from one side to the other but whose transition falls outside the segment endpoints has **not** crossed (it passed beside the line).

Trajectory assumptions you may rely on

These reflect the real-world behavior of vehicles approaching an intersection:

1. Points are sorted by `t` in ascending order; timestamps are unique.
2. The vehicle approaches the intersection from one side and either crosses or doesn't — i.e. the trajectory crosses the stop-line at most once.
3. Equivalently: the side of the stop-line on which trajectory points lie changes at most once along the trajectory. You may use this property algorithmically.

You may **not** assume:

- `y` is monotonic (the vehicle may wiggle in image coordinates).
- The trajectory starts on one side of the line and ends on the other (it may not reach the line at all, or may end exactly on it).
- The stop-line is horizontal.

Computing the crossing time

If the trajectory crosses (per the definition above), return the timestamp of the crossing:

- If the stop-line transition occurs **at a sample** that lies on the segment, that sample's timestamp is the crossing time.
- If the stop-line transition occurs **between two samples** on opposite sides of the line, interpolate linearly between those two samples to estimate the crossing time.

Return `None` if the vehicle never crosses the segment — including when the path only reaches the line without proceeding to the far side, or when it crosses the **infinite** extension of the stop-line outside the

segment endpoints.

Examples

#	Stop-line	Trajectory (t, x, y)	Expected	Why
1	(5, 100) – (15, 100)	(10, 7, 90), (11, 8, 95), (12, 7, 98), (13, 7, 102)	12.5	Side changes between t=12 and t=13; interpolated on the segment
2	(5, 100) – (15, 97)	(10, 7, 90), (11, 8, 95), (12, 7, 98)	None	Trajectory ends before reaching the line
3	(5, 100) – (15, 97)	(10, 7, 90), (11, 8, 95), (12, 7, 98), (13, 7, 102)	12.35	Interpolated crossing on the tilted segment
4	(9, 97) – (19, 94)	(10, 7, 90), (11, 8, 95), (12, 9, 97)	None	Path only reaches the line and ends on it — it never proceeds to the far side, so there is no crossing
5	(5, 100) – (15, 100)	(10, 8, 102), (11, 8, 100), (12, 8, 98)	11.0	On-segment sample — the side change lands exactly on a sample lying on the segment (distance $0 \leq 1$ px), so that sample's timestamp is the crossing time, with no interpolation
6	(5, 100) – (15, 100)	(10, 20, 102), (11, 20, 98)	None	Crosses the infinite stop-line — at (20, 100), ~5 px beyond endpoint (15, 100) — but the finite segment is never crossed; the vehicle passed beside it
7	(5, 100) – (15, 100)	(10, 8, 101.5), (11, 8, 100), (12, 8, 101.5)	None	Graze and retreat — the center reaches the line at a single sample then returns to the same side; the trajectory never visits the far side, so there is no side change and no crossing
8	(5, 100) – (15, 100)	(10, 8, 100), (11, 9, 100.5)	ValueError	Two samples lie on the stop-line (distances 0 and 0.5 px); ≥ 2 on-line points violate the at-most-one-crossing guarantee → malformed input

Error / edge-case contract

Return `None` (do **not** raise) for:

- Empty trajectory.
- Single-point trajectory. A single point cannot establish a stop-line transition, so it never constitutes a crossing — regardless of whether that point lies on the segment.
- Trajectory entirely on one side of the line, or that only reaches the line without proceeding to the far side.
- Trajectory that touches the *infinite* line only outside the segment endpoints.

You **must raise** `ValueError` for malformed inputs:

- Stop-line with two identical points.
- `car_id` empty or non-string.
- Any negative coordinate in the stop-line or in any trajectory point. Image coordinates are non-negative by definition: `(0, 0)` is the top-left corner of the image, with `x` and `y` increasing toward the bottom-right. A negative value indicates malformed input, not a point outside the frame.
- A trajectory containing **more than one** point that is *on the stop-line* (within one pixel of the segment). A vehicle crossing at most once can graze the line for at most a single sample; two or more on-line points contradict the at-most-one-crossing guarantee and indicate corrupt or mislabeled input.

Vertical stop-lines (`x1 == x2`) **must be supported** — do not reject them.

Robustness — the processor must be total. Every input must produce either a valid `RLRResult` or a `ValueError`. No other exception may escape `process_request`. In particular, non-numeric, `NaN`, or infinite coordinates, and structurally malformed points (wrong arity, `None`) must be rejected with `ValueError` — they must not be allowed to leak a `TypeError` from deep in the geometry, nor to silently return a result.

Tests

Include a `unittest` test suite for `RLRRequestProcessor`. All of the worked examples above must pass. Beyond that, choose your own coverage — we're looking for tests that exercise the contract you committed to in your code, including degenerate inputs and the geometric edge cases you think matter. Quality and judgment over quantity.

Constraints

- Python 3.10+.
- **Standard library only**, including for tests — use `unittest`.
- Trajectories may contain up to **100,000 points**.

Deliverables

1. Runnable code, with a `main.py` that runs the worked examples and prints results. `main.py` must **also** demonstrate the unhappy path: it must feed at least one malformed request through the same loop. The driver loop is required to handle bad input gracefully — catch the `ValueError`, report it per request, and continue processing the remaining requests. A single bad request must never abort the run.
2. `unittest` test suite runnable via `python -m unittest` from the project root.
3. `README.md`: how to install, how to run, any assumptions you made beyond this spec.

4. `NOTES.md` (≤ 1 page): which AI tools you used, what you accepted as-is, what you rewrote, and why.

What we evaluate

- **Correctness** on the worked examples and on the edge cases you choose to handle.
- **Algorithmic choice**: trajectories may be long. Did your algorithm exploit the structural assumptions, or process every point regardless?
- **Use of stated assumptions**: did you over- or under-assume relative to what the spec actually guarantees?
- **Robustness**: is the processor total — valid result or `ValueError` for *every* input — and does the `main.py` driver survive bad input?
- **Tests**: do they cover the contract, or just the happy path?
- **Code quality**: readability, naming, type hints, no dead code.
- **AI fluency**: can you explain what you accepted, what you rewrote, and why.

Important. You are expected to **fully understand your solution end-to-end** and to be ready to defend your design choices in the follow-up interview. Part 2 will revisit your code and explore "what would you do if..." extensions to the problem.

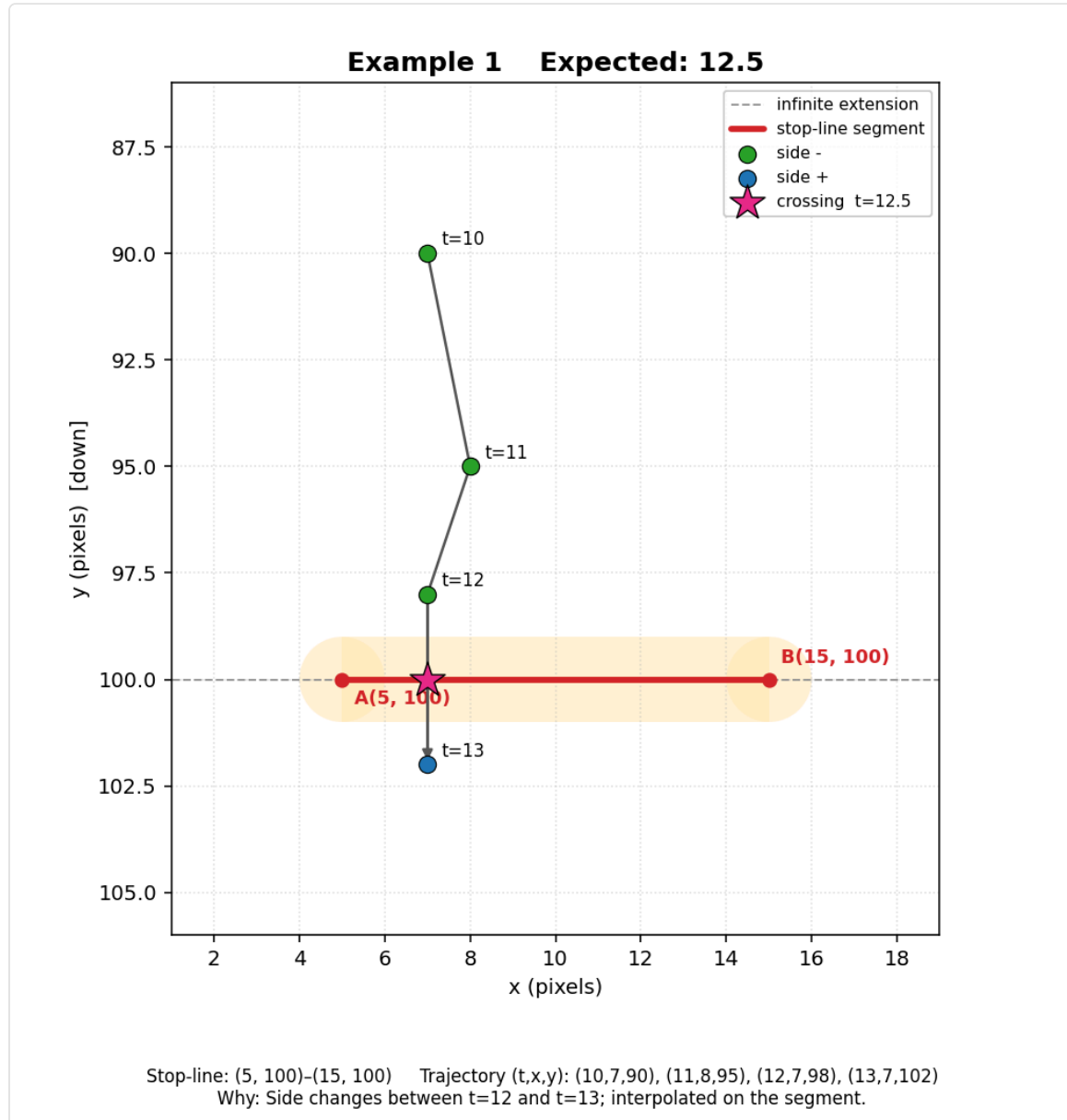
Suggested time: ~2.5 hours. If you're past 3.5 hours, stop and document what's left in `NOTES.md`.

Using AI tools

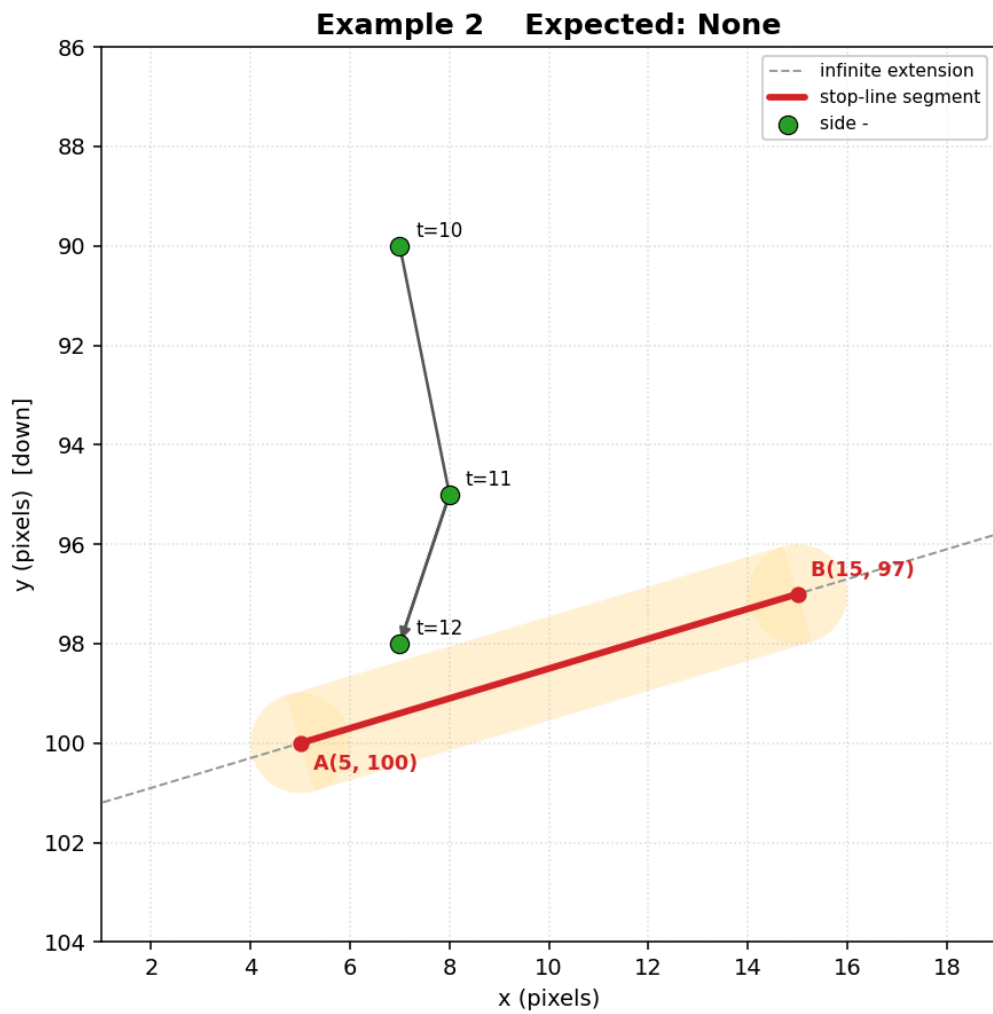
You are expected to use AI tools (ChatGPT, Copilot, Claude Code, Cursor, etc.), including features such as custom instructions or rules. Along with your submission, be ready to explain: how you used AI tools; which parts of the solution were AI-assisted; any non-trivial corrections or changes you made to AI output.

Appendix: Example diagrams

Each diagram is drawn in image coordinates (origin top-left, y increasing downward), with the stop-line segment in red, its infinite extension dashed, the shaded ± 1 -pixel "on-line" band, the trajectory coloured by side, and the crossing point where one exists.



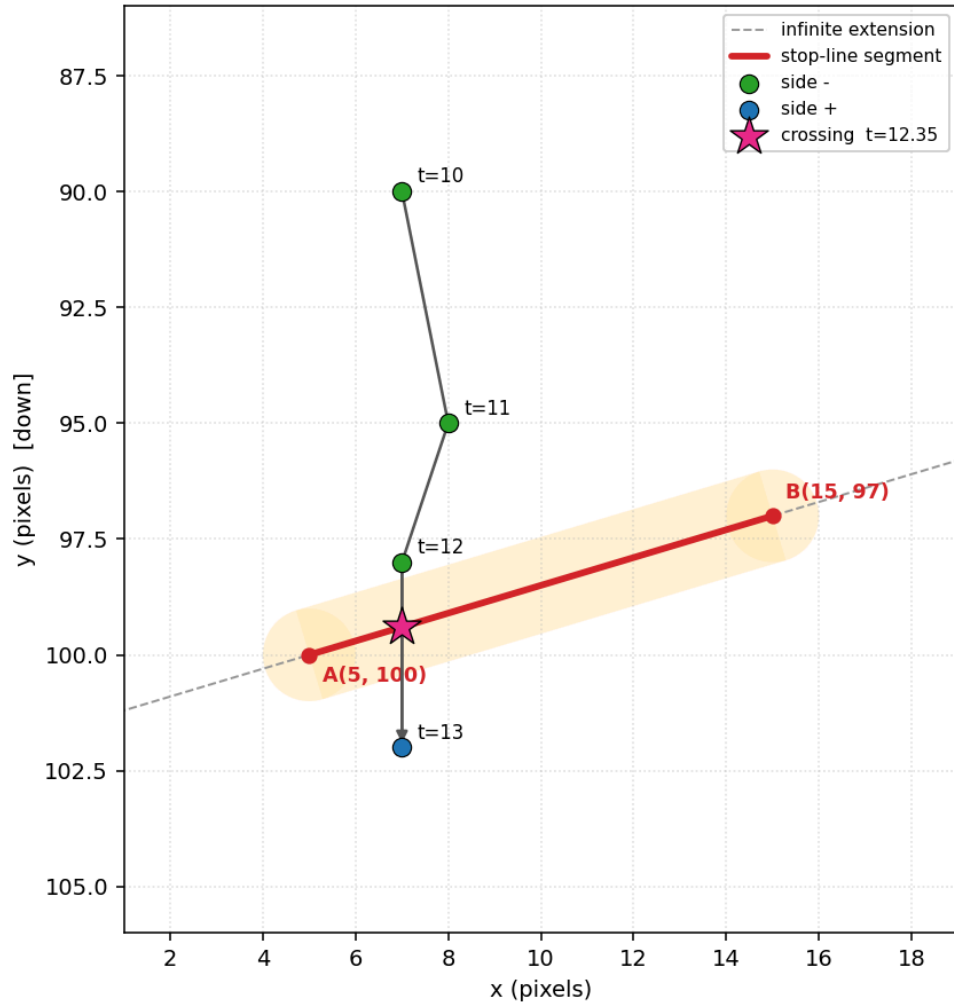
Example 1 — Expected: 12.5



Stop-line: (5, 100)–(15, 97) Trajectory (t,x,y): (10,7,90), (11,8,95), (12,7,98)
Why: Trajectory ends before reaching the line.

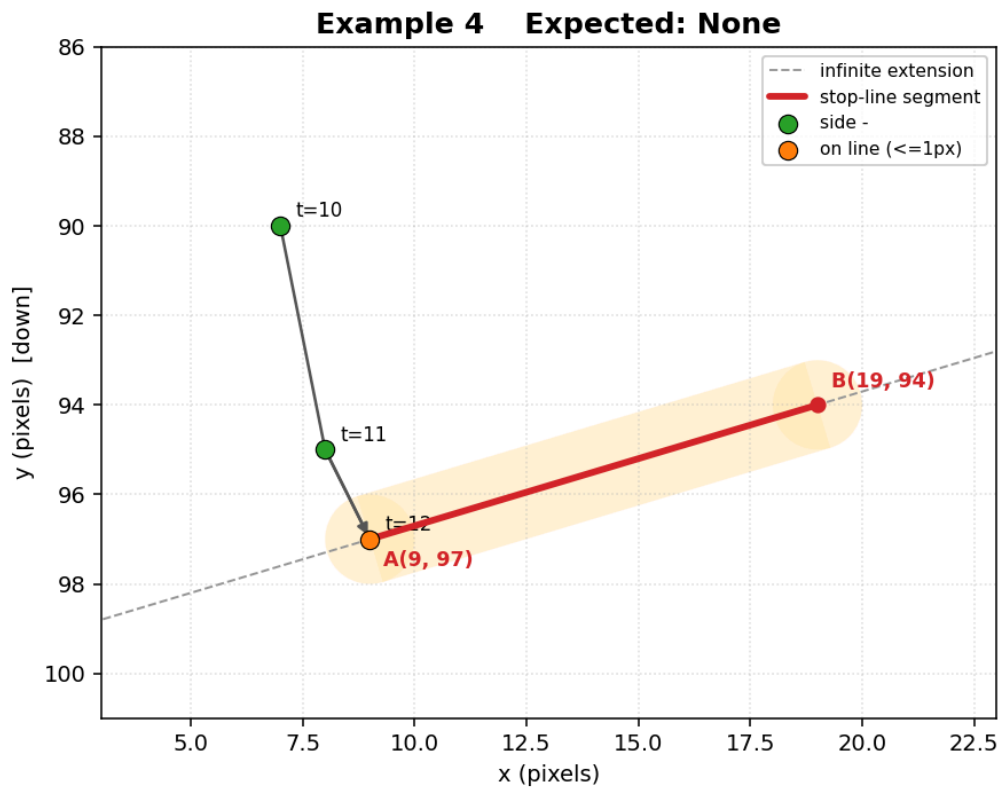
Example 2 — Expected: None

Example 3 Expected: 12.35

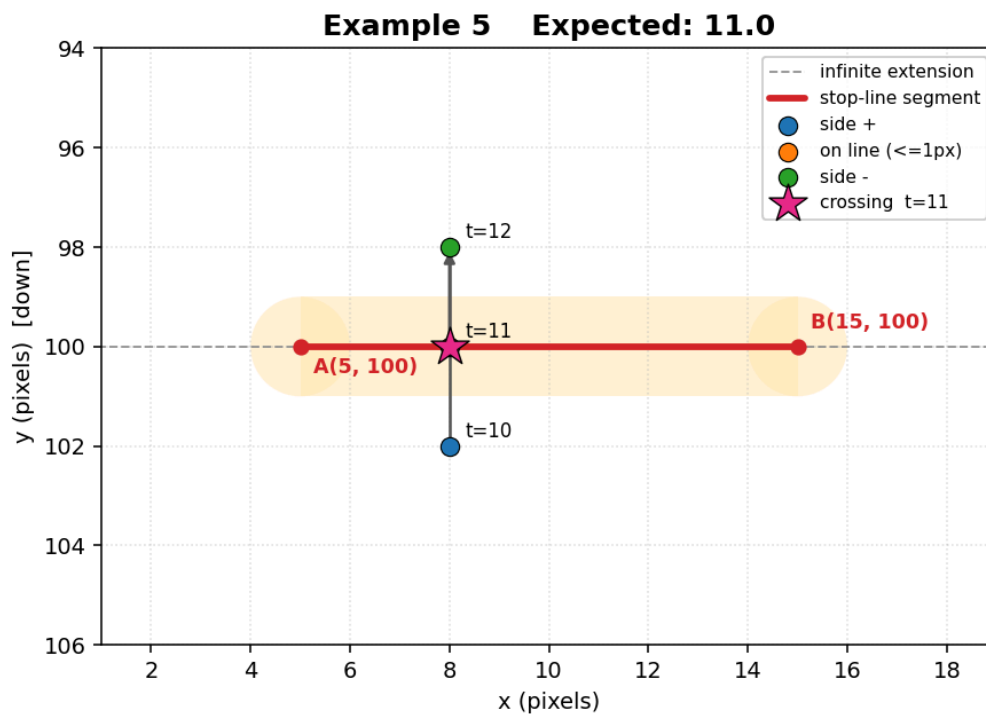


Stop-line: (5, 100)-(15, 97) Trajectory (t,x,y): (10,7,90), (11,8,95), (12,7,98), (13,7,102)
 Why: Interpolated crossing on the tilted segment.

Example 3 — Expected: 12.35

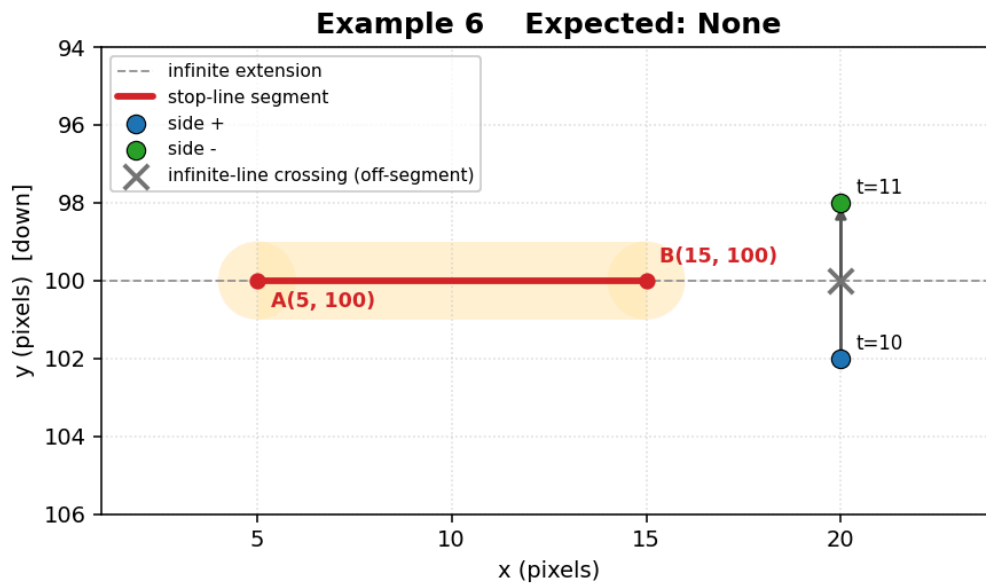


Example 4 — Expected: None



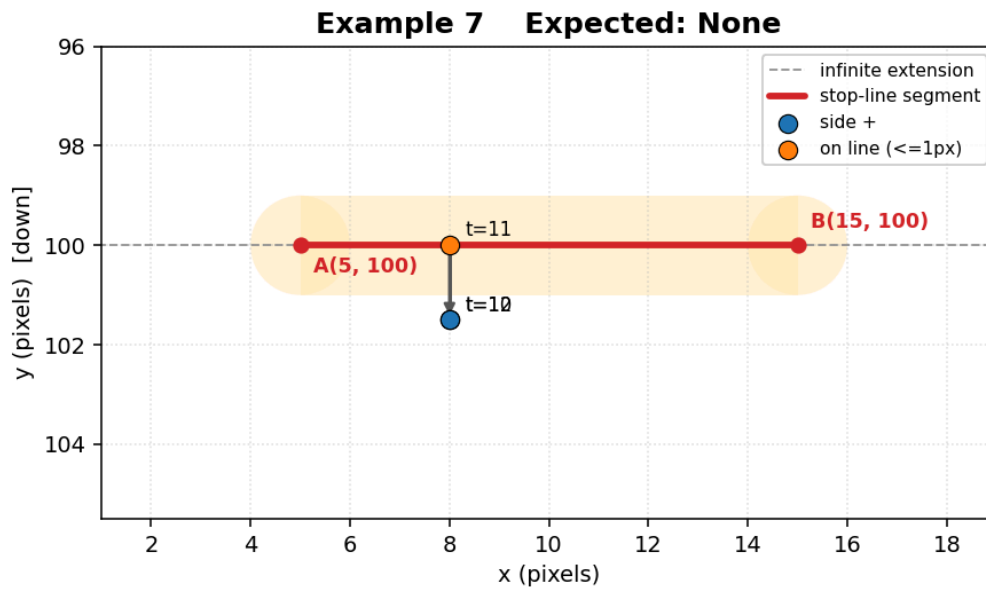
Stop-line: (5, 100)–(15, 100) Trajectory (t,x,y): (10,8,102), (11,8,100), (12,8,98)
 Why: On-segment sample: the side change lands on a sample lying on the segment (dist $0 \leq 1\text{px}$); its timestamp is the crossing time (no interpolation).

Example 5 — Expected: 11.0



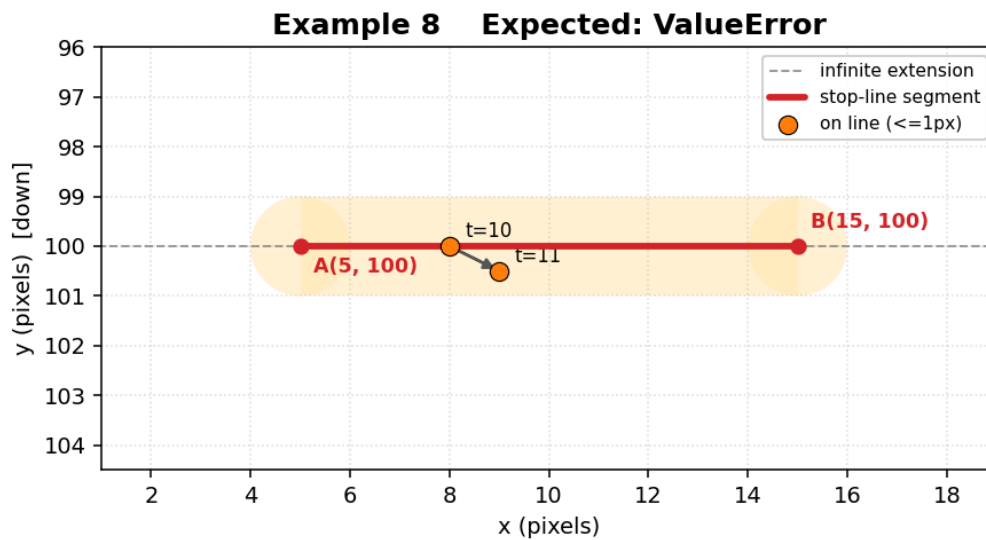
Stop-line: (5, 100)-(15, 100) Trajectory (t,x,y): (10,20,102), (11,20,98)
 Why: Crosses the INFINITE stop-line at (20,100), ~5px beyond endpoint B;
 the finite segment is never crossed - passed beside it.

Example 6 — Expected: None



Stop-line: (5, 100)-(15, 100) Trajectory (t,x,y): (10,8,101.5), (11,8,100), (12,8,101.5)
 Why: Graze & retreat: reaches the line at one sample then returns to the same side; the far side is never visited, so no side change and no crossing.

Example 7 — Expected: None



Stop-line: (5, 100)-(15, 100) Trajectory (t,x,y): (10,8,100), (11,9,100.5)
 Why: Two samples lie on the stop-line (dist 0 and 0.5px); ≥ 2 on-line points violate at-most-one-crossing $\Rightarrow$ malformed input.

Example 8 — Expected: ValueError